

Assignment 2: Coding Basics

Sean Burke

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
Fives <- c(1,5,10,15,20,25,30,35,40,45,50,55) #Creating a sequence from 1 to 55 counting by 5  
#2.  
mean(Fives) #calculating the mean of the number sequence
```

```
## [1] 27.58333
```

```
median(Fives) #determining the median of the sequence
```

```
## [1] 27.5
```

```
#3.  
pmax(mean(Fives), median(Fives)) #I am returning the larger of the two values, mean or median
```

```
## [1] 27.58333
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5.
Students <- c("John","Jane","Sean","Bruce") #vector type: string
Grades <- c(92,98,55,78) #vector type: numeric
Scholarship <- c(TRUE,TRUE,FALSE,FALSE) #vector type: boolean
#7.
student_info <- data.frame(Students,Grades,Scholarship)
#8.
names(student_info) <- c("Name","Grade","Scholarship")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A data set can contain information of many types but a matrix can only hold data of a single type. The columns in a data frame can also be named to categorize several types of data.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
Pass_or_Fail <- function(x){
  if(x < 50) {
    print("Fail")
  } else if(x > 50) {
    print("Pass")
  }
}
#11. Create a function using ifelse()
Pass_or_Fail2 <- function(x){
  ifelse(x < 50, "Fail", "Pass")
}
#12a. Run the first function with the value 52.5
Pass_or_Fail(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5  
Pass_or_Fail2(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
Test_Results <- c(45,90,67,19,78)  
#Pass_or_Fail(Test_Results)  
#13b. Run the second function with the vector of test scores  
Pass_or_Fail2(Test_Results)
```

```
## [1] "Fail" "Pass" "Pass" "Fail" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: Only the `ifelse` worked, the `if...else` did not. This is because `if...else` statements can only print results of one input at a time, not a whole vector's worth. So because these operations are vectorized, meaning every input is being run at once, a regular `if..else` statement cannot handle the multiple inputs, so only the `ifelse` statement can work.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)